JCP PAPER CAMPAIGN

Research Update

Tunable coordinate-network bank with off-mesh quadrature · viscous Burgers 2D and 3D

23 October 2026

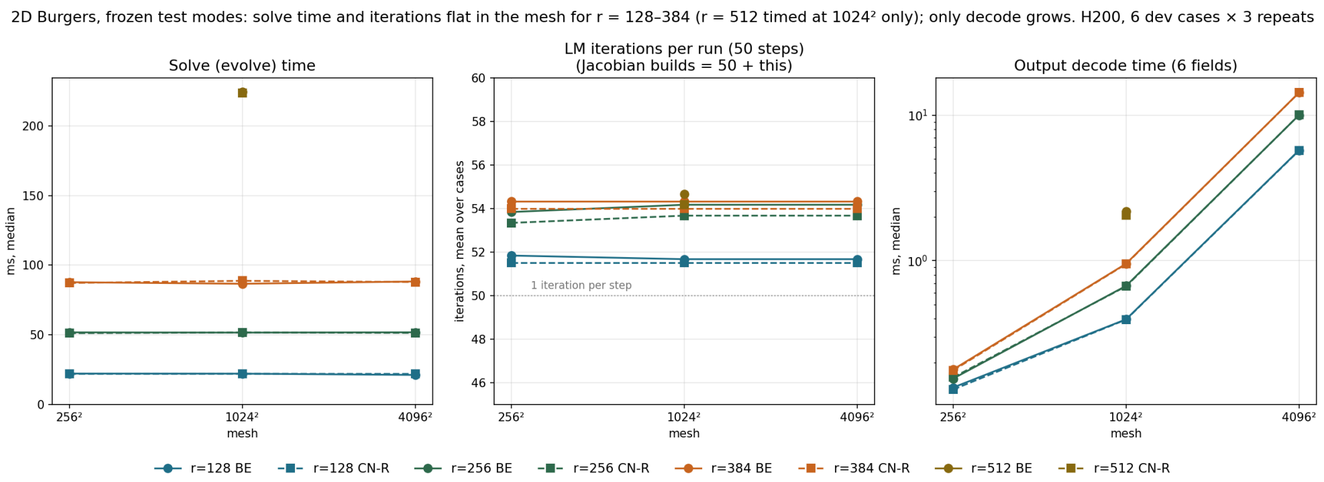
PART 1

The dial and the mesh

One trained bank; choose how many shapes to use; the solve does not depend on the mesh.

QUESTION Do iterations or decoding depend on the mesh?

Iterations are flat; only writing the output grows

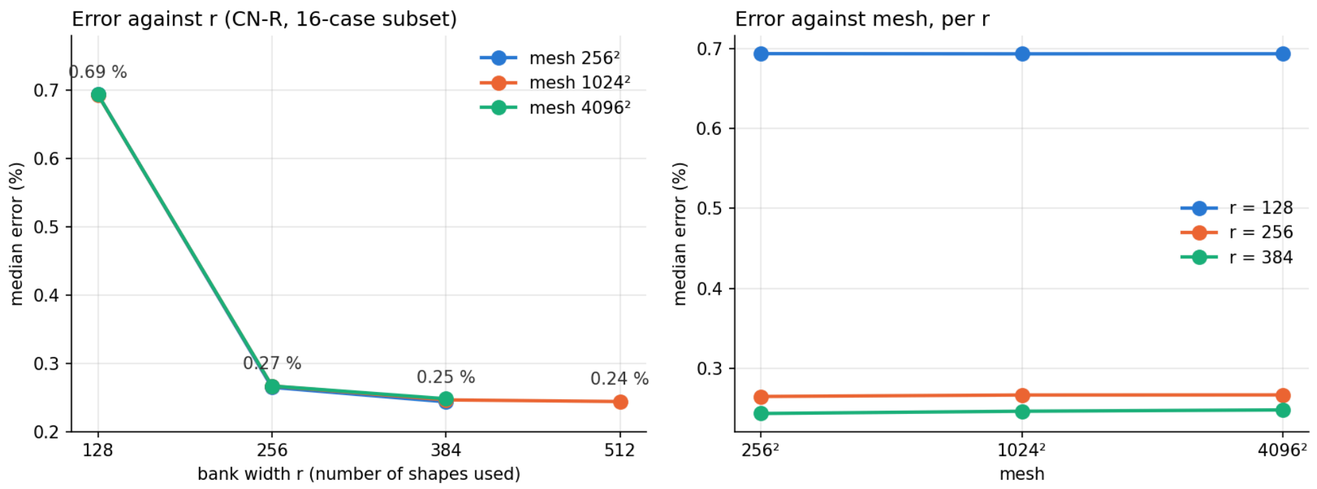


- **Solve:** r 128 ≈ 22 ms, r 256 ≈ 52 ms, r 384 ≈ 88 ms at every mesh
- **Iterations:** 51.5–54.3 Levenberg–Marquardt iterations per 50 steps, same on every mesh; at most 5 in any step
- **Decode:** 0.13–0.18 ms at 256² → 5.7–14.3 ms at 4096²
- Solid BE, dashed CN-R: schemes overlap

P1 timing job 5352971 (H200), frozen test modes, 6 dev cases × 3 repeats; Codex-checked · 5

QUESTION How do we get a dial r from one network?

Reorder the 512 shapes so the first r are the best r

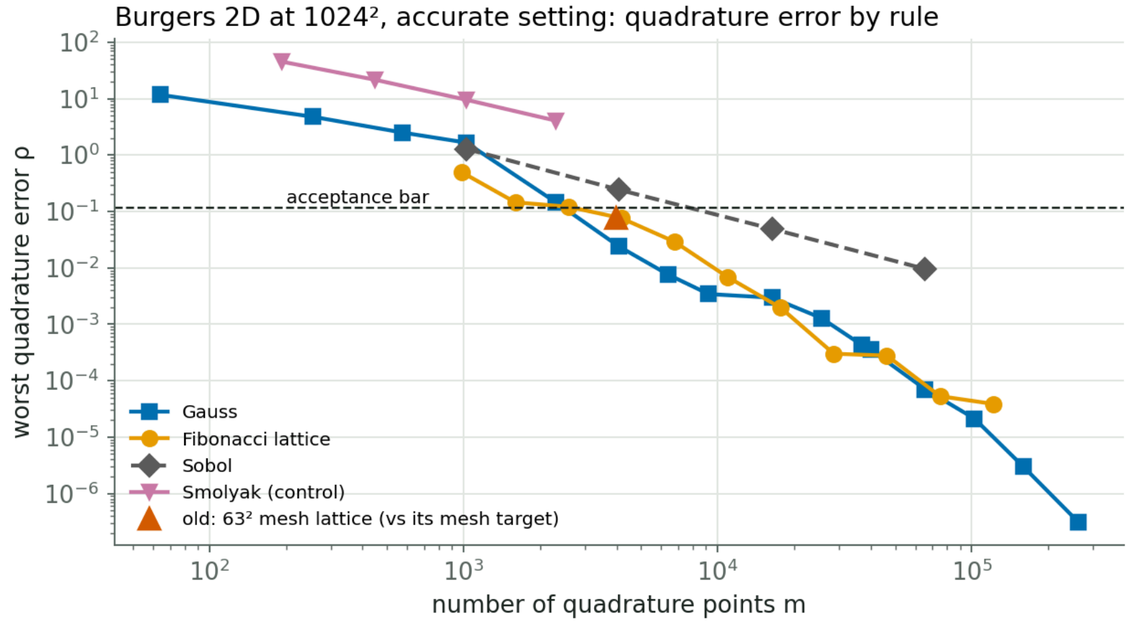


- QR of the shapes on a grid makes them orthonormal
- SVD of all training coefficients sorts directions by how much they are used
- Result: one rotation T; the first r columns are the best r-shape model
- Median error (CN-R, 16 cases): r 128 → 0.69 %, 256 → 0.27 %, 384 → 0.25 %, 512 → 0.24 %; flat after 256 (the time step takes over)

make_rotation.py; P1 analysis.json, 16-case subset (figs/make_error_vs_r_fig.py) · 3

QUESTION How many quadrature points does the nonlinear term need?

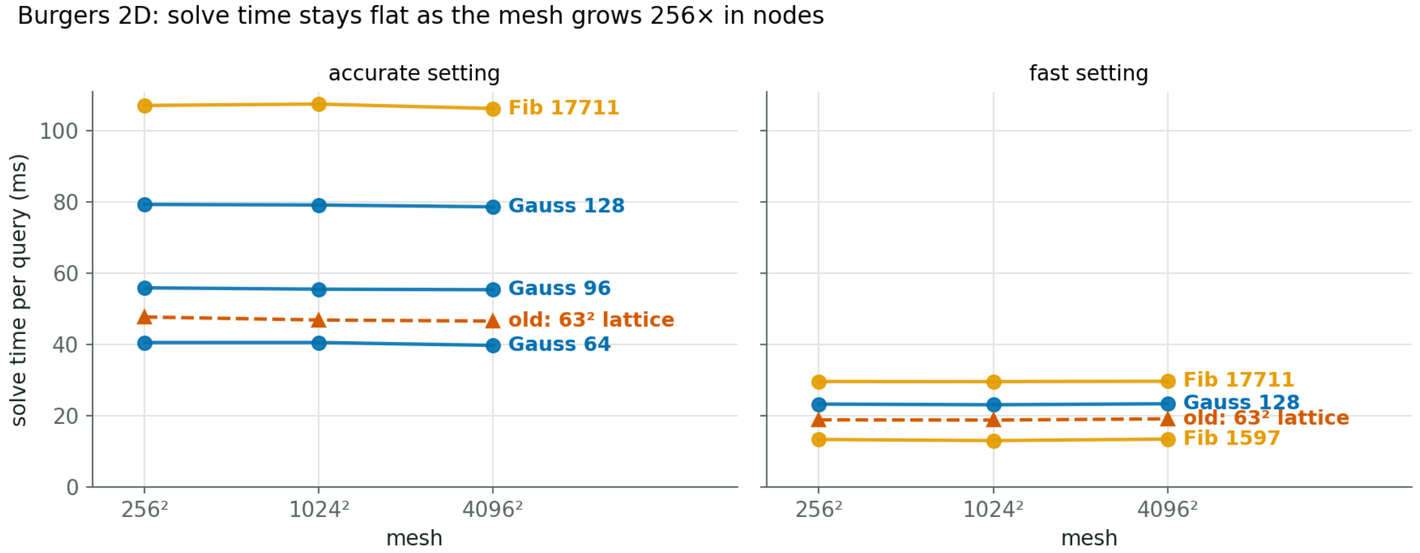
Gauss and Fibonacci rules pass with a few thousand points; Sobol and Smolyak need far more



- ρ: worst relative error of the quadrature nonlinear term against a converged reference (Gauss 640²), over every state the model reaches
- Acceptance bar: ρ ≤ 0.116
- At about 4,000 points: Gauss 0.019, Fibonacci 0.033
- Sobol converges slowly; Smolyak (a control) never reaches the bar
- The old 63² mesh lattice (triangle) is measured against its own mesh target, so it is not directly comparable

QUESTION What do the quadrature points cost, and does that depend on the mesh?

Cost grows with the number of points, never with the mesh



- **Accurate:** r = 384, 1536 tests; **fast:** r = 128, 512 tests
- Every rule is flat from 256² to 4096²: ratio 0.94–1.06 over all 40 measured arms
- More points cost more: Gauss 64² (4,096) → 96² (9,216) → 128² (16,384) → Fibonacci 17,711
- At 4096²: Gauss 96² 54.5 ms against the old 63² lattice 46.3 ms (accurate); Fibonacci 1597 13.3 ms against 18.7 ms (fast)

Off-mesh quadrature study 2D, one H200 job, dev6 × 3 repeats; solve = query − decode · 6

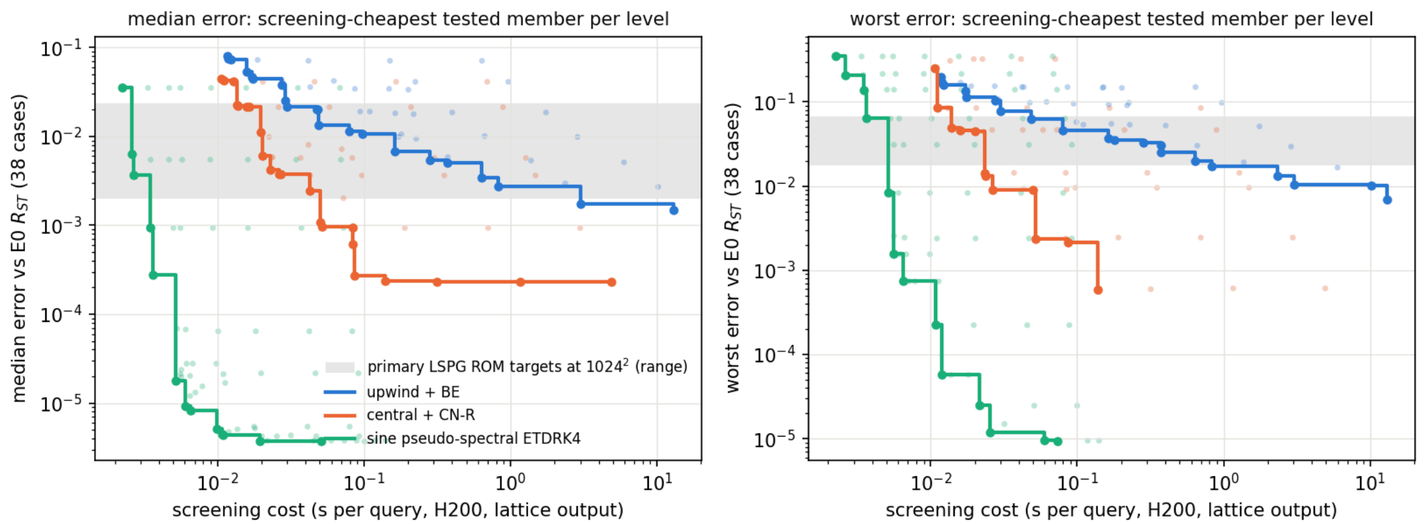
PART 2

Accuracy against speed

How our model compares with full solvers and with tuned POD reduced models, in 2D and 3D.

QUESTION In 2D, how cheaply do full solvers reach our accuracy?

A coarse spectral solver reaches our accuracy band at far lower cost

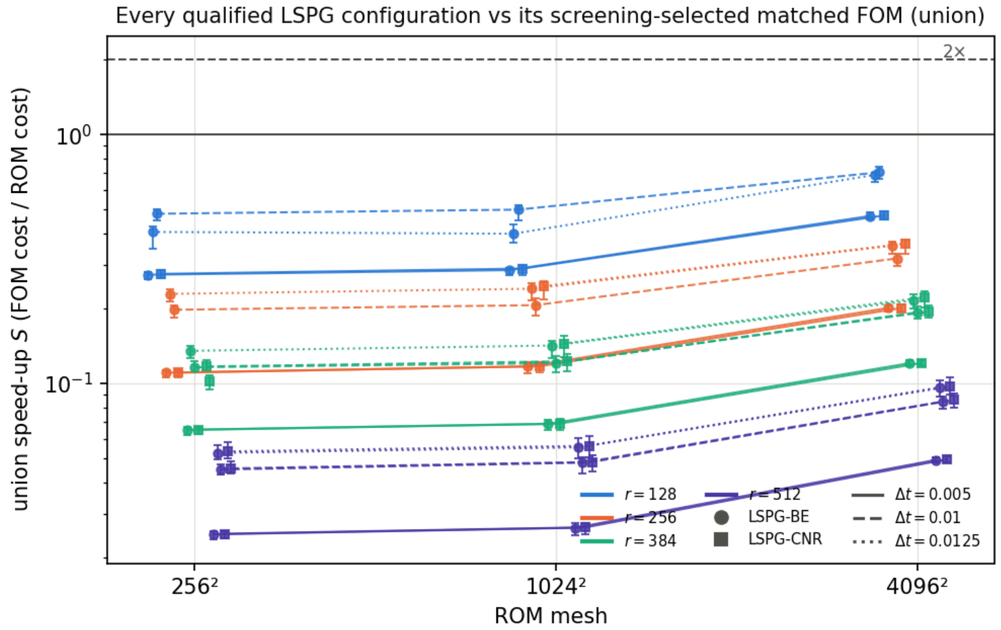


- Grey band: the accuracy our LSPG models reach at 1024^2 (median and worst)
- Green: sine pseudo-spectral with ETDRK4 reaches that band at the lowest cost
- Orange (central + CN-R) and blue (upwind + BE) need much more time
- Example at 4096^2: our r = 384 CN-R query 97 ms against 11.7 ms for the matched spectral solver

QUESTION In 2D, what is our speed-up at matched accuracy?

Slower

Below 1× for every configuration, closing as the mesh grows

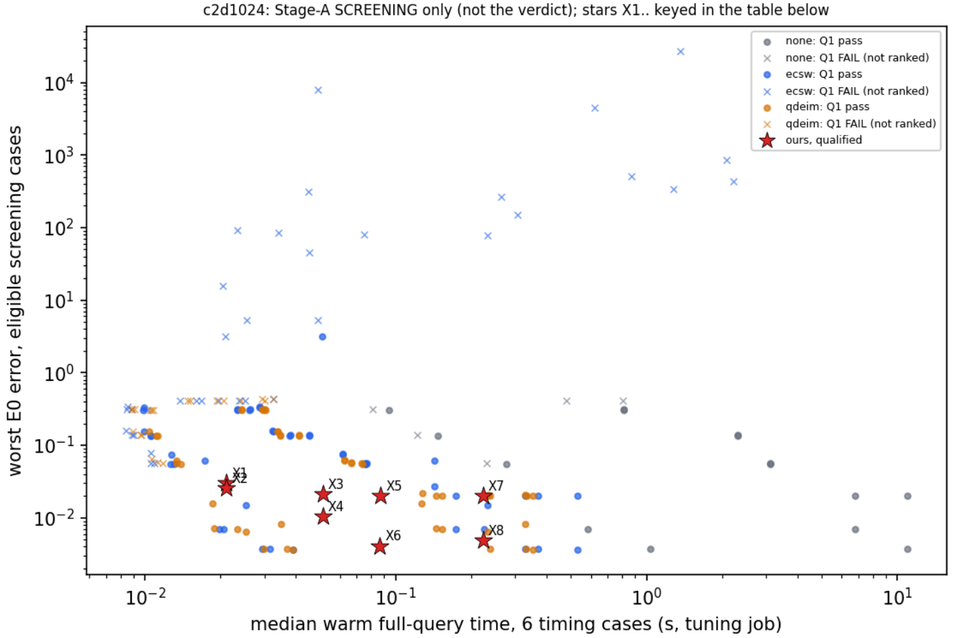


- Speed-up S = full-solver cost ÷ our cost at the same accuracy; above 1 means we are faster
- At 4096^2: S = 0.049–0.71 across 21 configurations; none reaches the 2× bar
- S rises with the mesh because our solve cost is flat while the full solver's grows
- Smaller r (blue, r = 128) is closest to break-even

QUESTION In 2D, how do we compare with a tuned POD reduced model?

Slower

POD with ECSW or Q-DEIM matches our accuracy in less time

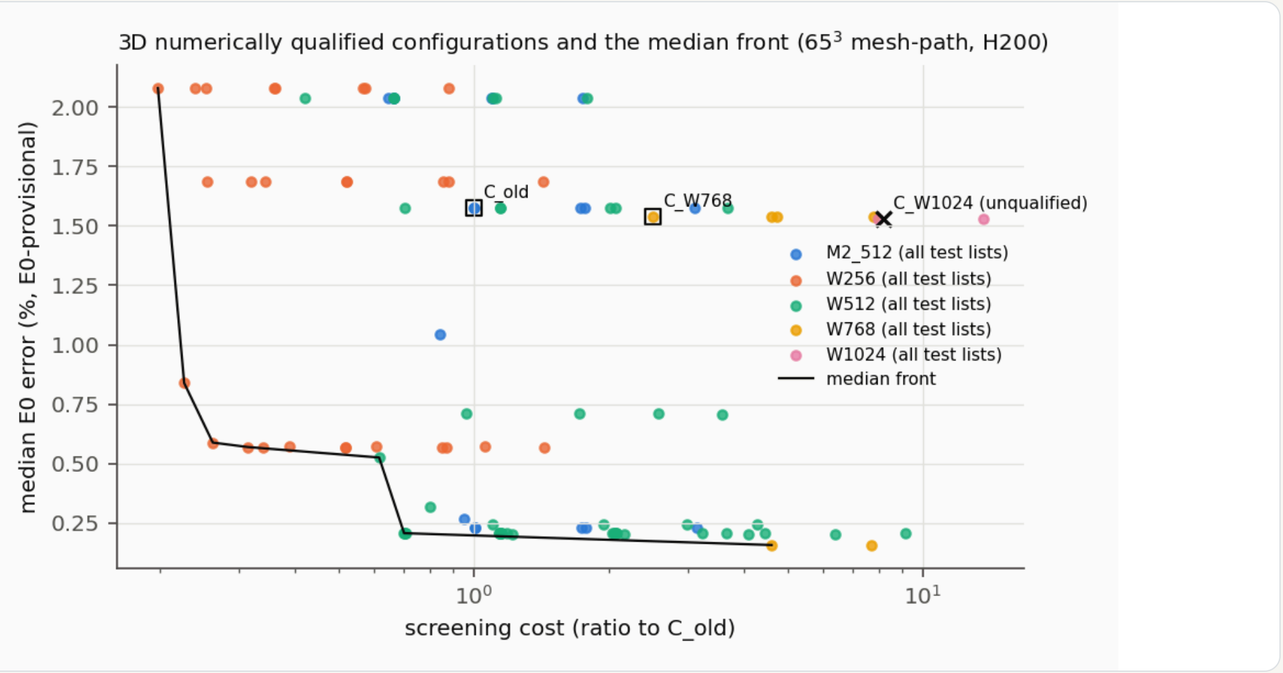


- Red stars: our qualified configurations; dots: POD + ECSW (blue), Q-DEIM (orange), none (grey)
- Dominated (faster and more accurate) in 8 of 8 configurations at 256^2 and at 1024^2
- Example: ours r = 384 CN-R 86.8 ms, worst 1.78 %; POD + ECSW 30.7 ms, worst 1.35 %

QUESTION In 3D, does the accuracy–cost dial work?

Works

Yes: wider banks buy accuracy, and the qualified model is 2.95× faster than our earlier one

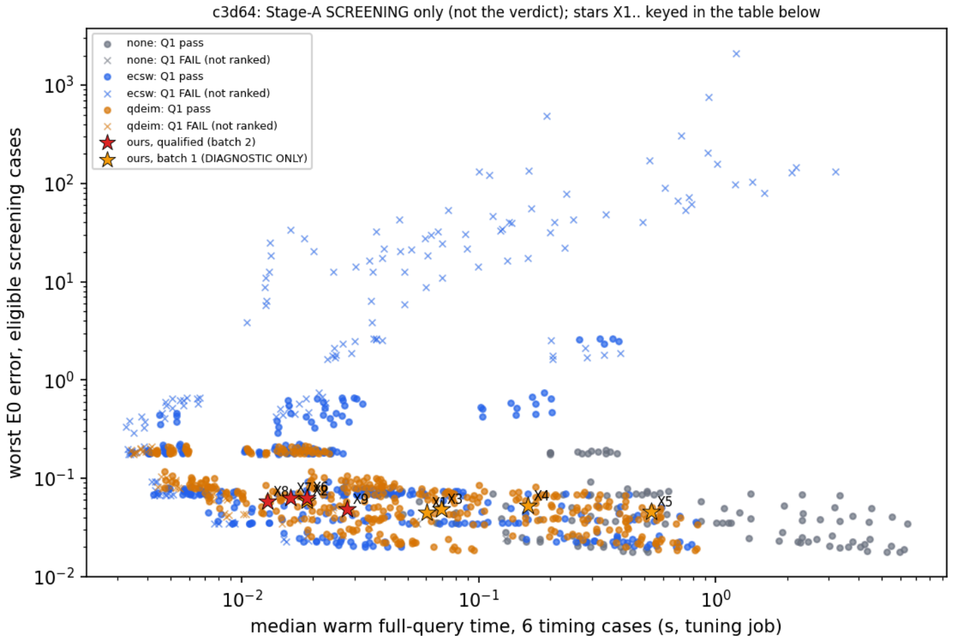


- Each dot is a numerically qualified 3D configuration (bank width, quadrature, solver)
- The black front: the cheapest configuration at each accuracy level
- Qualified speed-up 2.95× over our earlier wide model C_W768 at comparable accuracy
- Earlier unqualified claims (5.4×, 17×) are withdrawn

QUESTION In 3D, how do we compare with full solvers and POD?

Slower

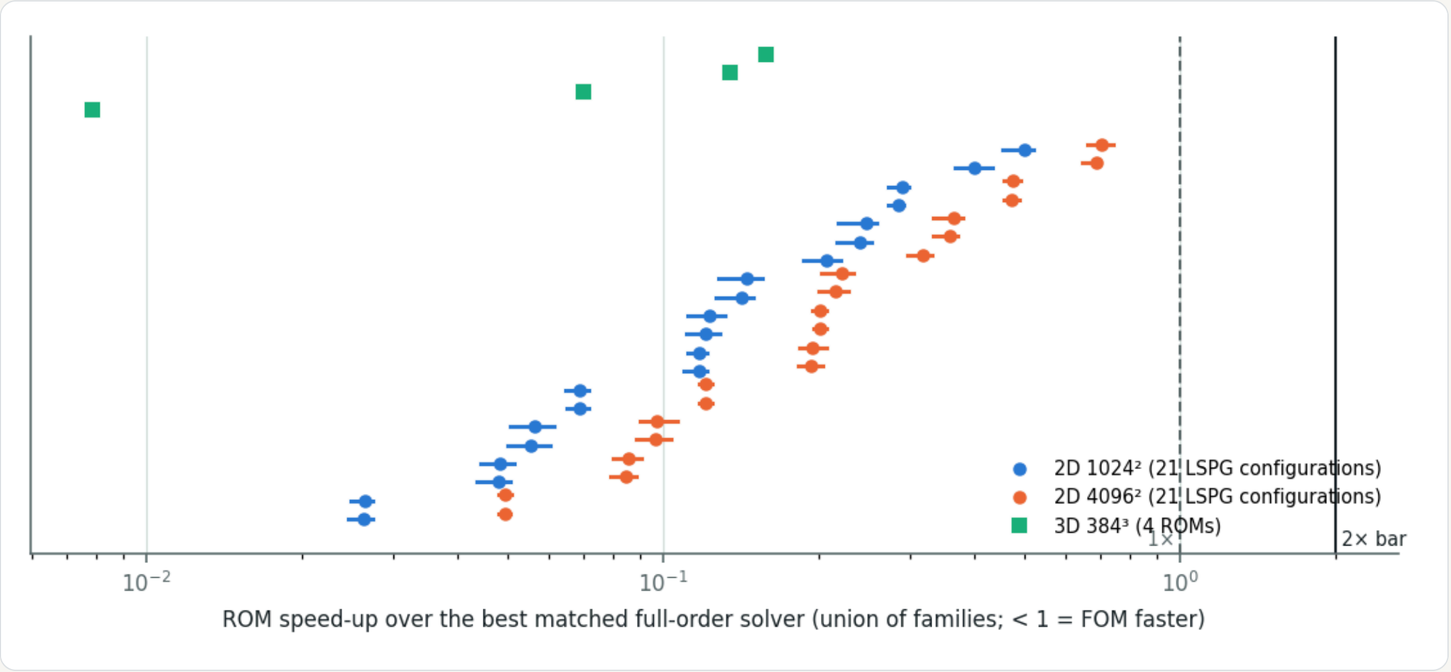
Both are faster: the full solver by 6–130×, POD in most cells



- Against the best matched full solver at 384^3: S = 0.0079–0.16 (4 models)
- Against POD at 65^3: dominated in 5 of 5 (diagnostic batch) and 2 of 4 (qualified batch)
- POD here failed its own basis-adequacy check, so its results carry that restriction

QUESTION All together: where do we stand against the best full solver?

Every configuration sits left of 1×; 3D is furthest behind

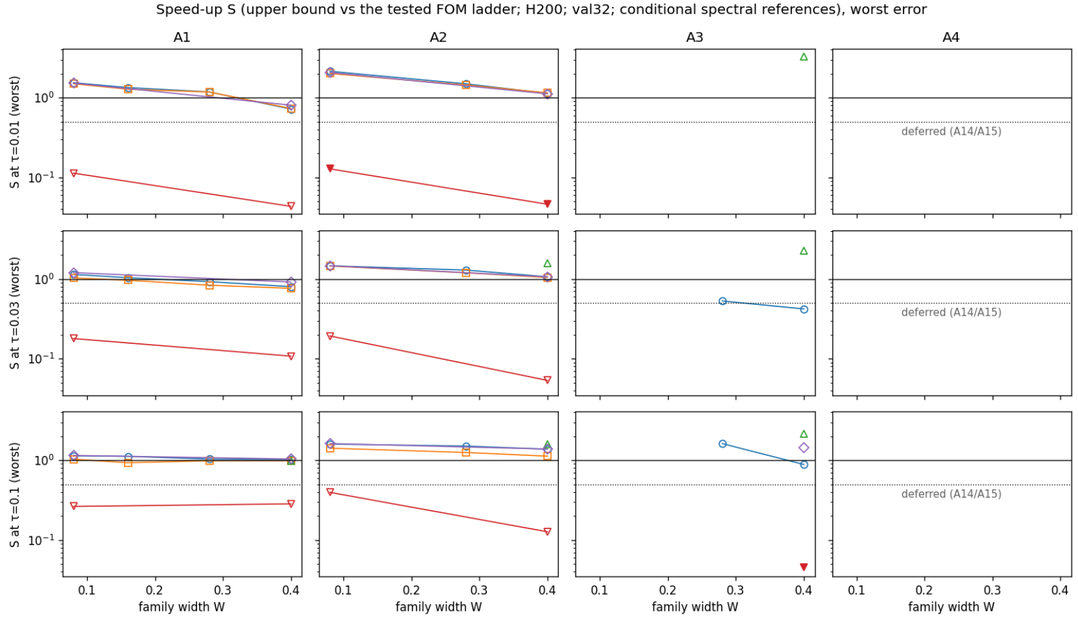


- Blue: 2D at 1024^2; orange: 2D at 4096^2; green: 3D at 384^3
- Dashed line: break-even (1×); solid: the 2× bar we registered
- 2D gets closer at the bigger mesh; 3D stays at 0.008–0.16

QUESTION Newest: does a faster time stepper change the picture?

New

With ETDRK4 the bank beats the tuned spectral solver by up to 2×, but so does POD



- Purple (our bank + ETDRK4): S = 0.81–2.07, above 1 in most cells
- Red (our bank, standard LSPG): S = 0.044–0.40
- Blue/orange (POD-Galerkin, POD + ECSW): up to 2.14 and 2.01, about the same as ours
- Best on narrow families (W = 0.08); falls as the family widens

QUESTION What does this add up to?

Mesh-independent and tunable; not yet faster than the best alternatives

Comparison	Where	Result
Mesh independence of cost and iterations	2D and 3D	Yes: flat within 1.01–1.09
Accuracy dial r	2D 128→512; 3D	Improves to r ≈ 256–384, then flat
Against the best full solver (deployed LSPG)	2D 4096^2; 3D 384^3	Slower: S 0.049–0.71; 0.0079–0.16
Against tuned POD + ECSW/Q-DEIM	2D 256^2, 1024^2; 3D 65^3	Slower: dominated 16/16 (2D), 7/9 (3D)
Against our earlier 3D model	3D 65^3	Faster: 2.95× (qualified)
Bank + ETDRK4 on the box (new)	2D box, narrow to wide	1–2× over spectral; level with POD (provisional)